

Ejemplo 2: Obtener Posts de una API con Fetch API y Promesas

Objetivo del ejercicio

- Practicar la obtención de **datos de tipo lista (posts)** desde un servidor.
- Utilizar **promesas (then, catch, finally)** para manejar la respuesta y errores.
- Mostrar los datos de forma dinámica en la página web.

Código completo (HTML + Bootstrap + JS)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo 2 - Fetch API Promesas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

  <h1 class="mb-4">Ejemplo 2: Obtener Posts con Fetch API</h1>
  <p>Haz click en el botón para cargar posts desde la API pública
<code>JSONPlaceholder</code>.</p>

  <button id="btnCargarPosts" class="btn btn-success">Cargar
Posts</button>

  <div id="resultadoPosts" class="mt-3"></div>

  <script>
    const btnCargarPosts =
document.getElementById('btnCargarPosts');
    const resultadoPosts =
document.getElementById('resultadoPosts');

    btnCargarPosts.addEventListener('click', () => {
      // Mensaje inicial de carga
      resultadoPosts.innerHTML = `<div class="alert alert-
info">Cargando posts...</div>`;

      // Petición fetch
      fetch('https://jsonplaceholder.typicode.com/posts')
        .then(response => {
          if (!response.ok) {
            throw new Error(`Error HTTP:
${response.status}`);
          }
          return response.json();
        })
    })
```

```

        .then(posts => {
            let html = '';
            // Mostrar solo los primeros 10 posts para
simplificar
            posts.slice(0, 10).forEach(post => {
                html += `
                    <div class="card mb-3">
                        <div class="card-header">Post #${post.id}
- Usuario ${post.userId}</div>
                        <div class="card-body">
                            <h5 class="card-
title">${post.title}</h5>
                            <p class="card-text">${post.body}</p>
                        </div>
                    </div>`;
            });
            resultadoPosts.innerHTML = html;
        })
        .catch(error => {
            resultadoPosts.innerHTML = `<div class="alert
alert-danger">Error: ${error.message}</div>`;
        })
        .finally(() => {
            console.log("Petición finalizada");
        });
    });
</script>

</body>
</html>

```

Explicación detallada

1. **Botón “Cargar Posts”**: Permite al alumno iniciar la petición.
 2. **fetch('https://jsonplaceholder.typicode.com/posts')**: Llama a la API para obtener los posts.
 3. **then(response => response.json())**: Convierte la respuesta en JSON.
 4. **then(posts => ...)**: Recorre los primeros 10 posts y los muestra en **tarjetas de Bootstrap**.
 5. **catch(error => ...)**: Captura errores de red o problemas en la API.
 6. **finally()**: Se ejecuta siempre y puede usarse para limpieza o logs.
-

Preguntas de comprensión para alumnos

1. ¿Por qué usamos `slice(0, 10)` al recorrer los posts?
 2. ¿Qué método de Bootstrap usamos para mostrar cada post de forma atractiva?
 3. ¿Qué pasa si la API devuelve un error 404 o 500?
 4. ¿Cómo podrías modificar el código para mostrar todos los posts sin límite?
 5. ¿Cómo podrías filtrar los posts por un `userId` específico antes de mostrarlos?
-

Mejoras sugeridas para el alumno

1. Añadir **un campo de búsqueda por título** para filtrar posts.
2. Implementar **paginación** para mostrar 5 o 10 posts por página.
3. Añadir **botones para expandir y ver el cuerpo completo** de cada post.
4. Mostrar **post en un modal** al hacer click en el título.
5. Hacer la petición **dinámica usando input de usuario** para filtrar posts por `userId`.